Module Guide for FitFolio

Team #1, Burn n' Earn
Davis Lenover
Preston Grewal
Angad Chhabra
Farid Bastoros
Zaid Alsaadi

November 11, 2026

1 Revision History

Date	Version	Notes
Nov 5, 2026	1.0	Davis Lenover: Revision −1. Sections 1 to 3, 5 Module Hierarchy, 7 Module Decomposition for the kernel, market and trading, and 9 Use Hierarchy Between Modules.
Nov 9, 2026	1.1	Davis Lenover: Email Service (M25) in 5 Module Hierarchy. 7 Module Decomposition: Users, Wallet and Email Service, a rationale for each module, and module health in Control and Plugins. Review fixes to Database, Market, Trading (queued trades and the simulation notice at Revision 0) and JSON Format, and to 9 Use Hierarchy Between Modules.
Nov 10, 2026	1.2	Farid Bastoros: 4 Anticipated and Unlikely Changes.
Nov 10, 2026	1.3	Davis Lenover: 2.1 Abbreviations and Acronyms, and review fixes to 7 Module Decomposition. Each market data provider gives its own request limit, so a new one changes only Market Providers.
Nov 10, 2026	1.4	Farid Bastoros: 6 Connection Between Requirements and Design.

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
API	Application Programming Interface
DAG	Directed Acyclic Graph
DPoP	Demonstrating Proof of Possession
FitFolio	A phone app where physical activity earns virtual currency that the user invests in a simulated stock market
FR	Functional Requirement (SRS)
H	Hazard (HA), such as H35
HA	Hazard Analysis
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
LC	Likely Change (SRS)
M	Module
MG	Module Guide
MIS	Module Interface Specification
NFR	Nonfunctional Requirement (SRS)
OS	Operating System
PoC	Proof of Concept
REST	Representational State Transfer
RFC	Request for Comments, an internet standard
SC	System Constraint (SRS)
SEC	Security Requirement (SRS and HA)
SRS	Software Requirements Specification
UC	Unlikely Change
ULC	Unlikely Change (SRS)

Contents

1 Revision History **i**

2 Reference Material **ii**
2.1 Abbreviations and Acronyms . . . ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**
4.1 Anticipated Changes . . . 2
4.2 Unlikely Changes . . . 4

5 Module Hierarchy **5**

6 Connection Between Requirements and Design **7**

7 Module Decomposition **10**
7.1 Hardware Hiding Modules . . . 10
7.1.1 Database Module (M1) . . . 11
7.1.2 Clock Module (M2) . . . 11
7.1.3 Market Providers Module (M3) . . . 11
7.1.4 Email Service Module (M25) . . . 12
7.2 Behaviour-Hiding Modules . . . 12
7.2.1 Control Module (M6) . . . 13
7.2.2 Constants Module (M7) . . . 13
7.2.3 Users Module (M8) . . . 13
7.2.4 Wallet Module (M9) . . . 15
7.2.5 Market Module (M10) . . . 16
7.2.6 Trading Module (M12) . . . 16
7.3 Software Decision Modules . . . 17
7.3.1 Plugins Module (M22) . . . 17
7.3.2 JSON Format Module (M23) . . . 17

8 Traceability Matrix **18**

9 Use Hierarchy Between Modules **19**

10 User Interfaces **22**

11 Design of Communication Protocols **23**

12 Database Design **23**

13 Timeline **23**

14 Generative AI Use Disclosure **23**
14.1 AI Tools Used . . . 23
14.2 How and Where Used . . . 23
14.3 Verification of AI-Generated Content . . . 23

List of Tables

1 Module Hierarchy . . . 7
2 Trace Between Requirements and Modules . . . 18
3 Trace Between Anticipated Changes and Modules . . . 19
4 Modules each module uses . . . 22

List of Figures

1 Use hierarchy among modules. Blue: hardware-hiding. Purple: behaviour-hiding. Orange: software decision. A dashed outline marks a module built after the proof of concept. . . . 21

3 Introduction

FitFolio joins two things that are usually kept apart: fitness and investing. Activity recorded on the phone earns virtual currency, the user spends it on simulated stocks priced from a real market, and the results feed goals, learning content and a social feed. That brings three design challenges:

- The features depend on each other. A user buying a stock needs the latest price from the market and their balance from the wallet, and must change both the balance and the shares together. Activity rewards land in the same wallet. The social feed shares activity and milestones, but must never show a balance or health data.
- Some parts rely on outside services that may change: the market data provider, which allows users to buy shares in the first place, and the health platforms on iOS and Android.
- Given this is a group project, the system will be developed in parallel, in stages, from the proof of concept to Revision 1.

When we first designed the backend, we routed commands to different services and let those services talk to each other directly, each with its own libraries. Taking a step back, this greatly increased coupling: a logic change in any one of these modules could easily ripple through the others. We needed a way to separate the modules, so that each one keeps its data and its decisions to itself, and the others reach it only through a small interface. This helps to support changes and additions to the modules.

Decomposing a system into modules is a commonly accepted approach to developing software. We decompose FitFolio by information hiding: each module hides a "secret", a design decision that is likely to change, such as the market data provider or a health platform. Separate modules also naturally become work assignments that people can develop in tandem. With a defined interface in between, it becomes much easier to work in parallel.

FitFolio is two programs: a server, written in Python with FastAPI and backed by PostgreSQL, and a phone app, written in React Native. The phone app talks to the server over HTTPS with JSON. This MG covers the modules of both programs.

Our design follows three rules:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed. The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules. The remaining sections cover the user interfaces, the communication protocols, the database design and the timeline.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

The list covers every likely change in the SRS (SRS LC1 to LC5) and adds the design and implementation choices the team expects to revisit. Each anticipated change is the secret of one module, which Table 3 names. SRS LC3 names three things that may change, so it is split here into the provider (AC3), the update frequency and asset list (AC10) and the limits that go with them (AC7).

AC1: The database server that stores FitFolio's data, and how the server connects to it, such as a move to a different hosted database.

AC2: The source of the current time, such as a fixed time in tests instead of the server's clock.

AC3: The market data provider and the form of its replies and errors. The provider may change if its request limits or cost prevent the planned use (SRS LC3).

AC4: The source of recorded activity and its interface: Apple HealthKit, Health Connect, and later a wearable device read directly, with the extra data it gives, such as workout duration and calories burned (SRS LC2).

AC5: The phone operating system's services that the app relies on, such as secure storage, notifications and background work, which differ between iOS and Android and change with new versions of each.

AC6: The set of modules that make up the running server. Each stage, from the proof of concept to Revision 1, adds modules.

AC7: The values of the SRS symbolic constants (SRS Appendix C), such as the earning rates, the daily currency cap and the price staleness limit. The staleness limit would change with the price update frequency (SRS LC3).

AC8: How the server knows who is calling: the built-in demo user in the proof of concept, an email address and a password at Revision 0, and possibly other ways to sign in later.

AC9: How a balance is stored, changed and recorded, such as a new reason for a change when selling is added.

AC10: How often prices are fetched, which assets can be traded, and when the market counts as open. The update frequency and the asset list may change with the provider's request limits (SRS LC3).

AC11: The rules that turn recorded activity into currency, such as which activity types earn, how an implausible total is refused, and how the daily cap is applied.

AC12: The trade rules. The proof of concept only buys while the market is open. Selling, queuing a trade while the market is closed, and the simulation notice check come at Revision 0.

AC13: The rules for goals, streaks, rest days and milestones.

AC14: The learning content, the knowledge checks and the rules that decide when investment feedback is given.

AC15: What a user can share and with whom. Fitness challenges and opt-in leaderboards are a stretch goal, and would first need an agreed change to the SRS, which now prohibits ranking users (SRS LC5).

AC16: The number and selection of historical market scenarios, and where their past prices come from, which depend on the availability and cost of historical data (SRS LC4).

AC17: How consent, data export and account deletion are carried out, which may change with the law or with the app stores' health-data policies (SRS SC6).

AC18: The order of the app's screens and how the user moves between them.

AC19: The layout and appearance of each screen, which will change with the results of usability testing.

AC20: When and how the app sends activity data to the server. If the health platforms prevent reliable background updates, the app will update when the user opens it instead (SRS LC1).

AC21: When reminders are scheduled and how they are delivered.

AC22: What a server module hands to Control, such as a new kind of background work or health check.

AC23: The form in which amounts and refusals are sent to the phone.

AC24: How the app calls the server, such as its address, the session token it sends with each request, and when it retries.

AC25: The email delivery service and its interface.

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decision should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

The first three are the unlikely changes of the SRS (SRS Section 13), numbered there ULC1 to ULC3. The rest are decisions fixed at the architecture stage.

UC1: The currency is virtual and the investing is simulated, with no real money and no link to a bank or brokerage account (SRS ULC1, SC5). Real money would change the purpose of FitFolio and add payment, identity and regulatory modules.

UC2: Physical activity is the way a user earns currency (SRS ULC2). Activity, Wallet and Trading are all built around rewards that arrive from activity.

UC3: Users control what they share and with which friends, and a balance or raw health data is never shared (SRS ULC3).

UC4: FitFolio is two programs, a phone app and one server, and trading, market prices and the social feed need a connection to the server (SRS SC7). Moving this work onto the phone would change almost every module.

UC5: The server is one process with one relational database. A buy changes the balance and the shares, which two modules own, in one transaction (FR37). Splitting the server into separate services would mean redesigning how Wallet and Trading work together.

UC6: The server alone decides balances, rewards and trades (SEC-24). The phone only reports activity and asks for trades, so no module in the app holds these rules.

UC7: The app runs on Android phones and iPhones (SRS SC1) and is written once in React Native. A web or desktop version, or a separate native app for each platform, would replace every app module.

UC8: Activity recorded by the phone is read only through its health platform (SRS SC2). The app does not count steps itself or accept activity typed in by the user.

UC9: The app and the server communicate over HTTPS with JSON. The form of single values may change (AC23), but a different protocol would change every route and the API Service.

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

The hierarchy covers both programs in FitFolio, the server and the phone app. Modules that are only in the app are marked (app). Hardware-hiding modules hide the interfaces of software and devices the team does not write. These are the database server, the server's clock, the market data provider, the email service, and the phone's health platform and operating system. Behaviour-hiding modules hide the behaviour the SRS describes. Software decision modules hide choices the team made that the SRS does not mention.

M1: Database

M2: Clock

M3: Market Providers

M4: Health Service (app)

M5: Device Service (app)

M6: Control

M7: Constants (server and app)

M8: Users

M9: Wallet

M10: Market

M11: Activity

M12: Trading

M13: Goals†

M14: Learning†

M15: Social†

M16: Scenarios†

M17: Privacy†

M18: Navigation (app)

M19: Screens (app)

M20: Sync Service (app)

M21: Reminder Service (app)†

M22: Plugins

M23: JSON Format

M24: API Service (app)

M25: Email Service†

Modules marked with † in Table 1 are built after the proof of concept and are specified in the MIS at Revision 0.

Level 1	Level 2
Hardware-Hiding Module	M1 Database
	M2 Clock
	M3 Market Providers
	M25 Email Service†
	M4 Health Service (app)
	M5 Device Service (app)
Behaviour-Hiding Module	M6 Control
	M7 Constants (server and app)
	M8 Users
	M9 Wallet
	M10 Market
	M11 Activity
	M12 Trading
	M13 Goals†
	M14 Learning†
	M15 Social†
	M16 Scenarios†
	M17 Privacy†
	M18 Navigation (app)
	M19 Screens (app)
	M20 Sync Service (app)
	M21 Reminder Service (app)†
Software Decision Module	M22 Plugins
	M23 JSON Format
	M24 API Service (app)

Table 1: Module Hierarchy

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

Some requirements can only be met by a decision that the SRS leaves to the design. This section records those decisions, so that none of them is implicit. Each one names the

requirements it serves, by their number in the SRS, and the modules that carry it out. The *Rationale* field of each module in Section 7 gives the detail.

One server with one database. A trade must update the balance and the holdings together or not at all (FR37, NFR6), and two modules own that data. So the server is one process with one database, split into modules, and not separate services. M12 Trading pays through M9 Wallet in the same transaction of M1 Database, and only the route commits it, so a refused or interrupted trade saves nothing.

The server holds every rule about currency. Rewards, balances and holdings are calculated only on the server (SEC-24). The app sends daily totals for each activity type and nothing else (SEC-10), through M20 Sync Service. M11 Activity turns the totals into currency, with the plausibility limits and the daily cap (FR13, FR17, FR18). No app module holds a reward or trade rule, so a changed app cannot give itself currency.

One way to change a balance. Every change to a balance is recorded with its amount, time and reason, and the balance equals the sum of its changes (SEC-22). So M9 Wallet offers one call that changes a balance and records the change together, and refuses a change that would take the balance below zero (FR37). The caller must lock the portfolio first, so that two requests for one user take turns. That is how a repeated sync pays each day's activity once (SEC-19) and never more than the daily cap (FR18).

Each trade request has an ID. A trade submitted twice is recorded once (FR38, NFR6). So the app makes an ID for each trade request, and M12 Trading answers a repeated ID with the first result. A retry over a bad connection is then safe.

The server fetches prices on a schedule. Prices come from a third party, within its request limits (FR32, SC4). So M10 Market fetches each price once for all users, instead of each phone asking the provider. It keeps the time each price was obtained, marks a price that is too old (FR33), and M12 Trading refuses to trade on one while the market is open (FR39). The provider's key never leaves the server (SEC-33).

Each outside service has one interface. The market data provider may change (SRS LC3), activity is read only through each phone's health platform (SC2, SC3), and a password reset needs an email service (FR6). So each of these is hidden by its own module: M3 Market Providers, M4 Health Service and M25 Email Service. M5 Device Service does the same for the phone's operating system, so the rest of the app is the same on Android and iOS (NFR4). A change of service then stays inside one module (NFR3).

Sessions are stored on the server. Every request that reads or changes a user's data needs an authenticated session (SEC-15, NFR7). Users sign in with an email address and a password (FR1, FR4), and the server must be able to end a session on sign-out, on a password reset or after `SESSION_TIMEOUT` (FR5, FR6, SEC-17). So M8 Users stores sessions on the server instead of issuing self-contained tokens, limits sign-in attempts

(SEC-38), and gives every route the current user. The phone never sends a user ID. M24 API Service keeps the session token in the phone's secure storage, through M5 Device Service (SEC-39).

Amounts are exact decimals. Every balance and portfolio value must be accurate to within the tolerance the SRS sets, `VALUE_TOLERANCE` (NFR1, SEC-30). So every amount is an exact decimal with four decimal places on the server, and M23 JSON Format sends it to the phone as text, so that no precision is lost on the way.

Requests are checked before a route runs. A request with a missing, wrongly typed or out-of-range input is rejected (SEC-25). So each route declares its inputs, and a request that does not fit them is refused before the route runs, without sending back the values it sent. Every refusal a module raises itself has one shape, a code and a message, which M23 JSON Format defines. M24 API Service and M19 Screens then handle a refusal the same way for every module.

The tunable values are in one module. The SRS names its limits as symbolic constants (SRS Appendix C), so that a value can change without changing a requirement. M7 Constants holds them under the same names on the server and in the app, which keeps such a change small (NFR3).

Time is handed to the code. Many rules depend on the time: whether the market is open, whether a price is too old (FR33, FR39), and whether a daily total is dated after today in the user's time zone, in which case it is refused (SEC-24). So no module reads the system time. Each is handed M2 Clock, and M8 Users keeps each user's time zone. The phone puts each activity record on its calendar day before it sends the totals (SEC-23).

Features are added as plug-ins. The system is built in stages, and the effort of a likely change is limited (NFR3). So M6 Control builds the server from a list of modules, each described by a record that M22 Plugins defines. Adding a feature, such as Goals or Social, adds a module to the list and changes no existing module.

A scenario has its own portfolio. A market scenario starts with its own balance and no holdings, and must not affect the live portfolio (FR45, FR46, SEC-31). So M9 Wallet keeps a separate portfolio for each scenario, and M16 Scenarios trades in it through M12 Trading. The live balance is never read or changed by a scenario.

The social feed cannot reach a balance. Only the kinds of update the SRS lists can be shared, and a currency balance or a portfolio value is not one of them (FR65). So M15 Social uses Activity, Goals and Learning for what may be shared, and does not use Wallet or Trading (Section 9). A balance or a portfolio value cannot appear in an update, because Social has no way to read one. An individual activity record cannot appear either, since the app sends the server only daily totals (SEC-10, FR68). Social also offers no ranking of users (FR71).

Privacy rules are in one module. The user can download their data, delete their account and withdraw consent to read health data (FR9 to FR11, FR76, SEC-9, SEC-18). These rules change with the law and the app stores' policies (SC6), so M17 Privacy holds them, apart from the features whose data they act on.

What the user sees is in Screens alone. Several requirements are about display: every return and portfolio value is labelled as simulated (FR43), a gain or loss is shown by a sign or a text label and not by colour alone (FR44), and the core features stay usable at `MAX_TEXT_SCALE` text size (NFR8). So layout and wording are in M19 Screens and M18 Navigation, and the app's services hold none. Results of usability testing (NFR2) can then change a screen without changing a service.

Reminders are scheduled on the phone. Reminders are sent only in the periods the user chose, and contain no health data, balance or portfolio value by default (FR72 to FR75, SEC-34). So M21 Reminder Service schedules them on the phone, through M5 Device Service, with the user's settings.

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding". The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *FitFolio* means the module will be implemented by the FitFolio software.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented. The *Rationale* field says why the module is designed this way: the requirements it serves, by their number in the SRS (such as FR37 or SEC-15), and the design pattern it uses. Server paths under *Implemented By* are relative to `src/backend/app/`.

7.1 Hardware Hiding Modules

Secrets: The interfaces of software and devices the team does not write: the database server, the server's clock, the market data provider, the email service, and the phone's health platform and operating system.

Services: Serve as a virtual version of these outside systems for the rest of FitFolio, so that a change to one of them stays inside its module.

Implemented By: –

7.1.1 Database Module (M1)

Secrets: The database server and the connection to it.

Services: Opens sessions to the database, gives each request its own session, and provides the base that every module's tables are defined on. Each module still owns its own tables.

Rationale: A buy changes the balance and the shares, which two modules own, so both must be saved in one transaction (FR37). One database, reached only through this module, makes that possible, and is why the server is one process rather than separate services. Each request gets its own session, and only the route commits it, so a refused request saves nothing (NFR6). Naming every constraint lets a later migration change it by name.

Implemented By: FitFolio server, in `core/db.py`, with its settings in `core/settings.py`, using SQLAlchemy and PostgreSQL

Type of Module: Abstract Object

7.1.2 Clock Module (M2)

Secrets: The source of the current time.

Services: Gives the current time, with its time zone, to routes and background work. Tests replace it with a fixed time.

Rationale: Many rules depend on the time: when the market is open, when a price is out of date (FR33, FR39), and which day is today in the user's time zone, so a daily total dated later is refused (SEC-24). Code is handed the clock instead of reading the system time (dependency injection), so tests can set any time and check each rule.

Implemented By: FitFolio server, in `core/clock.py`, using the OS clock

Type of Module: Library

7.1.3 Market Providers Module (M3)

Secrets: The market data provider's API.

Services: Gets the latest quote for a stock through one interface that every provider follows, and picks the provider that `MARKET_PROVIDER` names. The Finnhub provider turns Finnhub's replies and errors into FitFolio's own quotes and errors. A fake provider makes up prices for development and tests.

Rationale: The prices come from a third party, within its request limits (SC4), and the provider may change if its request limits or cost prevent the planned use (SRS LC3). Every provider follows one interface (strategy), Finnhub's API is wrapped to fit it (adapter), and a setting picks one (factory), so a new provider, with its own request limit, changes only this module. Its key stays on the server (SEC-33). The fake provider lets the team develop and test without a key or an open market.

Implemented By: FitFolio server, in `market/providers/`, using Finnhub

Type of Module: Abstract Data Type, with one interface and a Finnhub and a fake implementation

7.1.4 Email Service Module (M25)

Secrets: The email delivery service's API.

Services: Sends an email to a user's address, such as a password reset link. A fake records each email instead of sending it, for development and tests.

Rationale: A user who forgets their password resets it through a link sent to their email address (FR6). The service is a third party the team may change, so, as in M3, every service follows one interface (strategy), the chosen one is wrapped to fit it (adapter), and a fake lets tests run without sending email. It is sent only the email address and the link, nothing else about the user (SEC-11), and its credentials stay on the server (SEC-33).

Implemented By: FitFolio server, at Revision 0

Type of Module: Abstract Data Type, with one interface, an implementation for the chosen service and a fake

7.2 Behaviour-Hiding Modules

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. This module serves as a communication layer between the hardware-hiding module and the software decision module. The programs in this module will need to change if there are changes in the SRS.

Implemented By: –

7.2.1 Control Module (M6)

Secrets: The set of modules that make up the running server.

Services: Builds the server from the list of module plug-ins. It adds each module's routes, and starts and stops each module's background work with the server. It also serves the health check, which reports the database and each module's own health.

Rationale: The server is a modular monolith. It is one process with one database, so a trade stays one transaction (FR37), and it is split into modules that each own their tables. Control is the small kernel that builds it from plug-ins (the plug-in pattern), so adding a module changes only the list in `registry.py`, never the code that builds the server. The health check shows when background work, such as the price poller, has failed even though no request saw an error.

Implemented By: FitFolio server, in `main.py`, `registry.py` and `core/routes.py`

Type of Module: Abstract Object

7.2.2 Constants Module (M7)

Secrets: The values of the SRS symbolic constants (SRS Appendix C).

Services: Gives every module the same values, such as the daily currency cap and the price staleness limit.

Rationale: The SRS names its tunable values as symbolic constants, such as the price staleness limit (SRS Appendix C), so a value can change without changing a requirement. One file on each side, server and app, with the same names, keeps each value easy to find and traceable back to the SRS.

Implemented By: FitFolio server, in `core/constants.py`, and a matching file in the app

Type of Module: Library, with exported types and constants only

7.2.3 Users Module (M8)

Secrets: How the server knows who is calling.

Services: Gives every route that needs one the current user, so the phone never sends a user ID. In the proof of concept, a request that needs a user comes from the built-in demo user while a setting turns it on, and is refused otherwise. Also adds users and keeps each user's record, such as their time zone.

Rationale: Every request that reads or changes a user's data must come from an authenticated user (SEC-15, NFR7). Routes get the current user from one place (dependency injection), so how users are known can change in this module alone, and the routes

stay the same. At Revision 0, users sign up and sign in with an email address and a password (FR1, FR4). The server keeps only a slow, salted hash of each password, so a stolen copy of the database gives away no password and makes guessing one slow (HA H45). After too many wrong passwords, an account refuses sign-ins for a while, even with the right password, so guessing a password by signing in is slow (SEC-38, HA H35). A forgotten password is reset through a link that M25 sends (FR6). An email can be read later or forwarded, so the link's random token works once and expires soon, which limits who could use it to take over the account (HA H35). The server keeps only the token's hash, so a stolen copy of the database can't be used to reset a password (HA H45).

Each sign-in starts a session that the server stores, and the phone keeps the session's random token in its secure storage (SEC-39) and sends it with every request. Sessions are stored on the server, not in self-contained tokens, because the server must be able to end one at once: on sign-out, on "sign out of all devices", after a password reset, or after `SESSION_TIMEOUT` without the user opening or using the app (FR5, FR6, SEC-17). The server keeps only a hash of each token, so a stolen copy of the database can't be used to act as a user (HA H45).

Between the phone and the hosting, the token only travels over HTTPS (SEC-16, HA H41), so no one on the network can read it unless the phone was set up to trust them. That isn't enough on its own, because whoever has a copy of the token can act as the user, and HTTPS protects it only while it travels. The phone and the server must both read the token to use it, so a copy can still be taken at either end, or by a proxy the phone trusts:

- on the phone, by malware on a rooted or jailbroken phone, or from a backup (HA H35)
- by a proxy an iPhone trusts. A debugging tool or a work phone's management profile can add the proxy's certificate, and the proxy then reads the traffic before passing it on. Android apps ignore such certificates unless the phone is rooted.
- on the server, or on a hosting proxy in front of it that ends HTTPS, if a log or crash report records the token by mistake

So each end guards the token too. Secure storage, set to keep the token on this phone only, keeps it away from other apps and stops a backup from moving it to another phone, unless the phone is rooted or jailbroken (SEC-39). The server keeps the token out of its logs and crash reports. It also gives the phone a new token from time to time, such as when the app opens, and retires the old one once the phone first uses the new one, so a lost reply doesn't sign the user out. After that, if the token was copied once, such as from a backup or a crash report, only one side holds the new token. So a retired token that comes back means the token was most likely copied, whichever side sends it. The server then ends the whole session, which stops the copy too. So a token copied once works at most until the real phone next asks for a new token, and

the server finds out if both keep using the session (HA H35). A proxy that stays in the path, malware that stays on the phone, or a log that keeps recording requests reads each new token too, so new tokens don't stop them. If the phone is lost and never used again, signing out of all devices ends the copy (SEC-17, HA H37). A new token doesn't count as user activity, so it never keeps an unused session alive past `SESSION_TIMEOUT` (SEC-17).

There is no separate encryption module. HTTPS comes with the hosting. The hosting signs the key exchange with the certificate's key, and the phone checks the certificate and that signature before it sends any data, so a man in the middle can't swap in their own keys unless the phone was set up to trust their certificate. HTTPS also makes new keys for each connection. A key exchange of our own would have to solve the same problems, and a simple one, where each side sends its public key in plain text, has no way to tell the real keys from a man in the middle's. Hashing and making random tokens are a few library calls inside this module. A later step could bind each session to the phone, as DPoP (RFC 9449) does. The app would make a key pair in the phone's hardware keystore, which never lets the private key out, and sign every request with it. The server would keep the public key with the session and refuse a request without a valid signature, so a copied token alone would be useless. It isn't in Revision 0 because the app and the server would both have to sign or check every request. The new tokens already limit a token copied once. A copier that stays needs a rooted or jailbroken phone, a proxy the phone was set up to trust, or a log that keeps recording the token.

Implemented By: FitFolio server, in `users/`

Type of Module: Abstract Object

7.2.4 Wallet Module (M9)

Secrets: How a portfolio's currency balance changes safely.

Services: Keeps each user's live portfolio and its balance, and records every change with its amount, reason and time. Changes a balance only through one call, which never takes it below zero and runs after the portfolio is locked, so that requests for one portfolio take turns. Gives other modules the balance and what a day's rewards have paid so far, and serves the balance to the phone.

Rationale: The server alone decides balances (SEC-24), a purchase that costs more than the balance is refused (FR37), and every change is recorded with its reason (SEC-22). One call changes the balance and records the change in the same transaction, which keeps the balance equal to the sum of its changes, and its check never lets the balance go below zero, even if two buys run at once. Locking the portfolio first makes requests for it take turns, so what a request reads stays true until it saves. That covers whether its request ID was used (FR38) and what a day's rewards have already paid, so a repeat

sync pays only the difference and a day never pays more than the daily cap (SEC-19, FR18). The call only accepts what the lock returns (a lock token), so skipping the lock is hard to do by accident. The live portfolio is kept apart from any market scenario's (SEC-31).

Implemented By: FitFolio server, in `wallet/`

Type of Module: Abstract Object

7.2.5 Market Module (M10)

Secrets: The live market data: the asset list, the latest prices, how old each price is, and when the market is open.

Services: Keeps every tradable stock's price up to date while the market is open, and marks a price as delayed when it is too old. Gives other modules the assets and their current prices, and serves the asset list to the phone, and the provider's usage to the team.

Rationale: Users trade at real prices (FR32), all in one currency (FR29, SEC-27). Fetching them on the server, once a minute while the market is open, keeps within the provider's request limits (SC4) and gives every user the same price. That rate may change with the provider (SRS LC3). While the market is open, a price older than the staleness limit is marked delayed (FR33), so Trading can refuse it (FR39). Other modules get read-only copies, never rows, so none can change a price by accident. The market's hours are part of this secret, not a module of their own, because they decide both when prices are fetched and when a price counts as delayed.

Implemented By: FitFolio server, in `market/`

Type of Module: Abstract Object

7.2.6 Trading Module (M12)

Secrets: The trade rules: buying, selling, holdings and portfolio value.

Services: Buys and sells whole shares at the current price (FR35, FR36), and queues a trade asked for while the market is closed, to run at the first price after it opens (FR40). Refuses a trade that is not allowed, such as one on a price that is out of date, without enough currency, or before the user has accepted the simulation notice (FR34). Applies each trade request once, and values the user's holdings (FR41, FR42).

Rationale: A trade changes the balance and the shares together, so Trading pays through Wallet in the same transaction as it saves the shares and the trade (FR37). A trade made while the market is open runs only at the price the user confirmed (SEC-29), and never on an out-of-date one (FR39). A queued trade runs at the first price after the open, which the user is told when they queue it (FR40). Each request carries an ID

from the phone (an idempotency key), so a retry over a bad connection never trades twice (FR38, NFR6). Values use exact decimals (NFR1). The proof of concept only buys, and refuses a buy while the market is closed. Selling (FR36), queuing (FR40) and the notice check (FR34) come at Revision 0.

Implemented By: FitFolio server, in `trading/`

Type of Module: Abstract Object

7.3 Software Decision Modules

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are *not* described in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 Plugins Module (M22)

Secrets: The shape of a module's plug-in.

Services: Defines what a module hands to Control: its name, its routes, its background work and its health check. Control can then add any module without knowing anything else about it.

Rationale: Control must be able to add any module without knowing anything else about it (the plug-in pattern). A plug-in is a small, read-only record of exactly what Control needs, so the code that builds the server depends on this shape and never on a module. Only the list in `registry.py` names the modules. A module with background work must include a health check, since that work can fail without any request noticing.

Implemented By: FitFolio server, in `core/plugins.py`

Type of Module: Record

7.3.2 JSON Format Module (M23)

Secrets: The JSON form of amounts and refusals.

Services: Sends amounts to the phone as text with four decimals, so no precision is lost, and sends every refusal a module raises in the same shape: a code and a message. A request that does not fit a route is refused in the web framework's own shape, without the values it sent.

Rationale: JSON numbers can lose exactness on the way, so amounts go as text with four decimals (NFR1). One refusal shape, with a code the phone acts on and a message it shows, keeps the phone's handling the same for every module. A request that does not fit a route is refused without sending back what it sent, since some values can't be written as JSON (SEC-25).

Implemented By: FitFolio server, in `core/money.py` and `core/errors.py`

Type of Module: Library

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Req.	Modules
R1	M??, M??, M??, M6
R2	M??, M??
R3	M??
R4	M??, M6
R5	M??, M??, M6, M??, M??, M??
R6	M??, M??, M6, M??, M??, M??
R7	M??, M??, M6, M??, M??
R8	M??, M??, M6, M??, M??
R9	M??
R10	M??, M??, M6
R11	M??, M??, M??, M6

Table 2: Trace Between Requirements and Modules

AC	Modules
AC1	M1
AC2	M2
AC3	M3
AC4	M4
AC5	M5
AC6	M6
AC7	M7
AC8	M8
AC9	M9
AC10	M10
AC11	M11
AC12	M12
AC13	M13
AC14	M14
AC15	M15
AC16	M16
AC17	M17
AC18	M18
AC19	M19
AC20	M20
AC21	M21
AC22	M22
AC23	M23
AC24	M24
AC25	M25

Table 3: Trace Between Anticipated Changes and Modules

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. Of two programs A and B, A *uses* B if correct execution of B may be necessary for A to complete the task described in its specification. That is, A *uses* B if there exist situations in which the correct functioning of A depends upon the availability of a correct implementation of B. Figure 1 illustrates the use relation between the modules. It can be seen that the graph is a directed acyclic graph

(DAG). Each level of the hierarchy offers a testable and usable subset of the system, and modules in the higher level of the hierarchy are essentially simpler because they use modules from the lower levels.

An arrow from module A to module B means that A uses B: the code of A imports B. To keep Figure 1 readable, one arrow into a group stands for uses of the modules in that group. The shared group holds the modules that many others use. For example, most server modules with routes use M8 Users to find the current user. Modules in the shared group can also use each other. Table 4 lists every use.

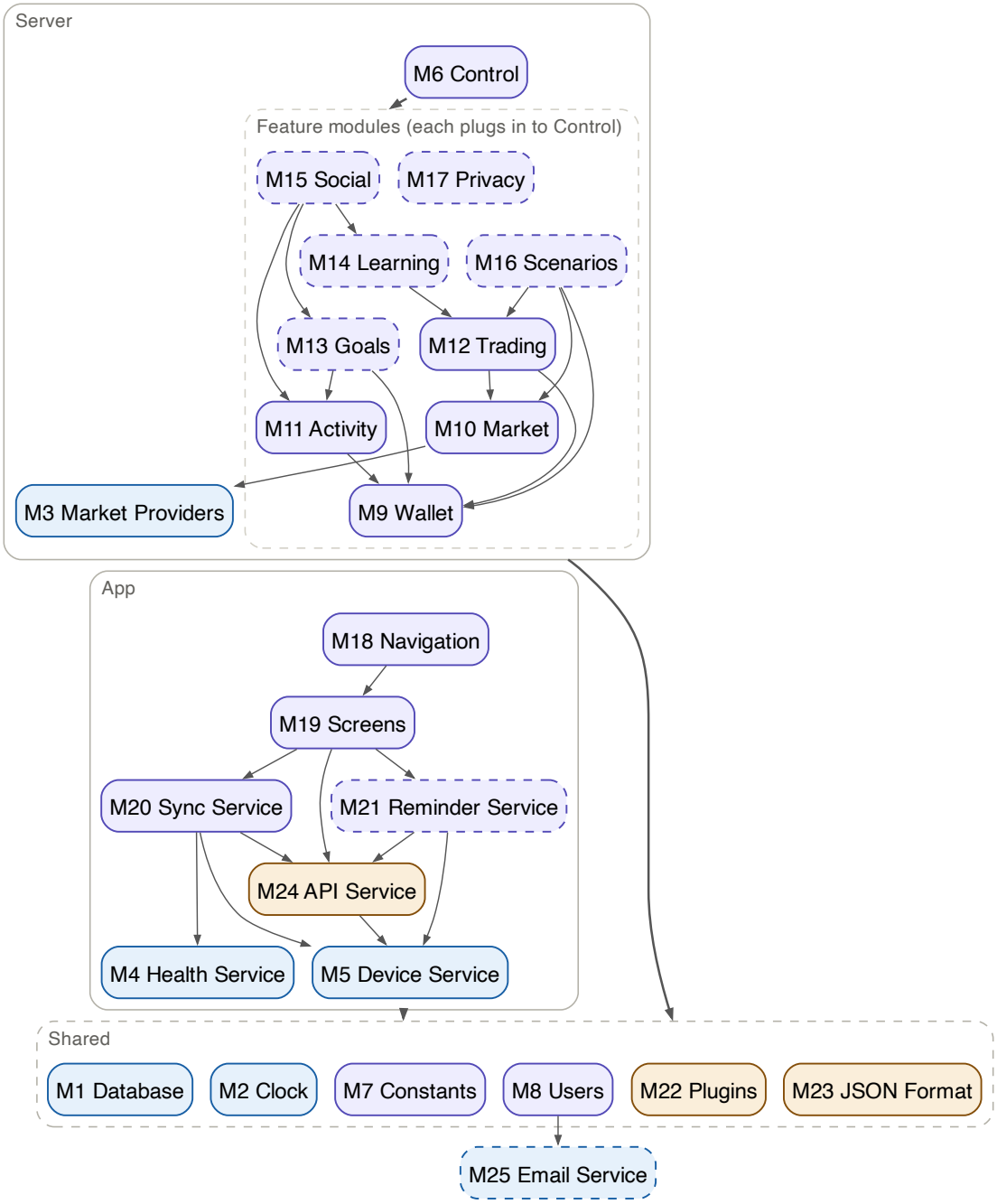


Figure 1: Use hierarchy among modules. Blue: hardware-hiding. Purple: behaviour-hiding. Orange: software decision. A dashed outline marks a module built after the proof of concept.

Module	Uses
M1 Database	None
M2 Clock	None
M3 Market Providers	M2
M4 Health Service	None
M5 Device Service	None
M6 Control	M1, M8, M9, M10, M11, M12, M13, M14, M15, M16, M17, M22, M23
M7 Constants	None
M8 Users	M1, M2, M7, M22, M23, M25
M9 Wallet	M1, M8, M22, M23
M10 Market	M1, M2, M3, M7, M22, M23
M11 Activity	M1, M2, M7, M8, M9, M22, M23
M12 Trading	M1, M2, M8, M9, M10, M22, M23
M13 Goals	M1, M2, M7, M8, M9, M11, M22, M23
M14 Learning	M1, M7, M8, M12, M22, M23
M15 Social	M1, M2, M8, M11, M13, M14, M22, M23
M16 Scenarios	M1, M7, M8, M9, M10, M12, M22, M23
M17 Privacy	M1, M8, M22, M23
M18 Navigation	M19
M19 Screens	M7, M20, M21, M24
M20 Sync Service	M4, M5, M7, M24
M21 Reminder Service	M5, M7, M24
M22 Plugins	None
M23 JSON Format	None
M24 API Service	M5
M25 Email Service	None

Table 4: Modules each module uses

10 User Interfaces

[Design of user interface for software and hardware. Attach an appendix if needed. Drawings, Sketches, Figma —SS]

11 Design of Communication Protocols

[If there is communication between hardware and software, it may be necessary to explain the design of the communication protocol. This section should be filled in with NA if it is not appropriate for a given project. —SS]

12 Database Design

[If a database is a significant component of the project, the database design should be summarized here. This section should be filled in with NA if it is not appropriate for a given project. —SS]

13 Timeline

[Schedule of tasks and who is responsible —SS]

[You can point to GitHub if this information is included there —SS]

14 Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

14.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

14.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

14.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.

- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]